

Cours Informatique

Programmation Langage C

SMP4

Pr.RAOUYANE Brahim

Contenu :

- ▶ **Séance 2 Operations :**
 - ▶ **Opérateurs** : binaire et unaire
 - ▶ **Conditions**
 - ▶ **Boucles itératives**
 - ▶ **Ruptures de séquence**

Instructions

Une **INSTRUCTION** simple est toujours terminée par un “;”.

- ▶ **Une instruction est :**
 - ▶ Soit une **instruction simple**,
 - ▶ Soit un **bloc**.
- ▶ **Une instruction simple est :**
 - ▶ Soit une instruction de **contrôle**
 - ▶ Soit une **expression** suivie de “;”.

Instructions

- ▶ **Un bloc a la structure suivante :**
 - 1) Une accolade ouvrante "{"
 - 2) Une liste de définitions locales au bloc (optionnelle)
 - 3) Une suite d'instructions
 - 4) Une accolade fermante "}".
- ▶ **Le terme instruction peut prendre de manière indifférente l'une des trois significations :**
 - ☐ **Instruction simple ;**
 - ☐ **Instruction de contrôle de flot ;**
 - ☐ **Bloc d'instructions.**

Instructions de contrôle: Condition

- ▶ Les **if** vérifie **si l'expression** de texte à l'intérieur des parenthèses **()** est **vrai** ou **faux**.
- ▶ Si l'expression de test est **vraie**, l'instruction à l'intérieur du corps d'if **est exécuté mais** si le test est **faux**, déclaration et instructions l'intérieur du corps de **si sont ignorés**.
- ▶ **Syntaxe:**

```
if (test expression) {  
    instruction 1;  
    instruction 2;  
    .....;  
    instruction n;  
}
```

Instructions de contrôle: Condition

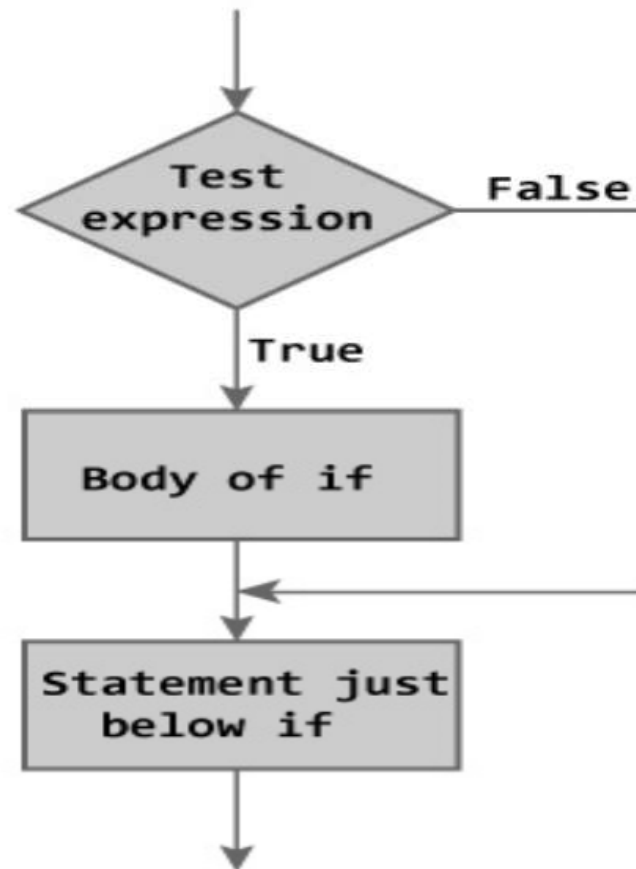


Figure: Flowchart of if Statement

Instructions de contrôle: Condition

► Exemple :

```
1  /*****
2      Prorgamme qui test si la valeur saisie par utilisateur
3      est une valeur nulle
4  *****/
5
6  #include <stdio.h>
7
8  void main()
9  {
10     int a;                //Declration de Variables
11     printf("Entrer la valeur de a :"); // Affichge de message sur ecran pour inviter utilisateur pour saisir la valeur de a
12     scanf("%d",&a);        // Lecture aprtir de Clavier
13
14     if(a==0)              // La condition avec if entre () , une seule instruction
15         printf("Attention la valeur est nulle");
16
17 }
18
```

► Exécution :

```
Entrer la valeur de a : 0
Attention la valeur est nulle
```

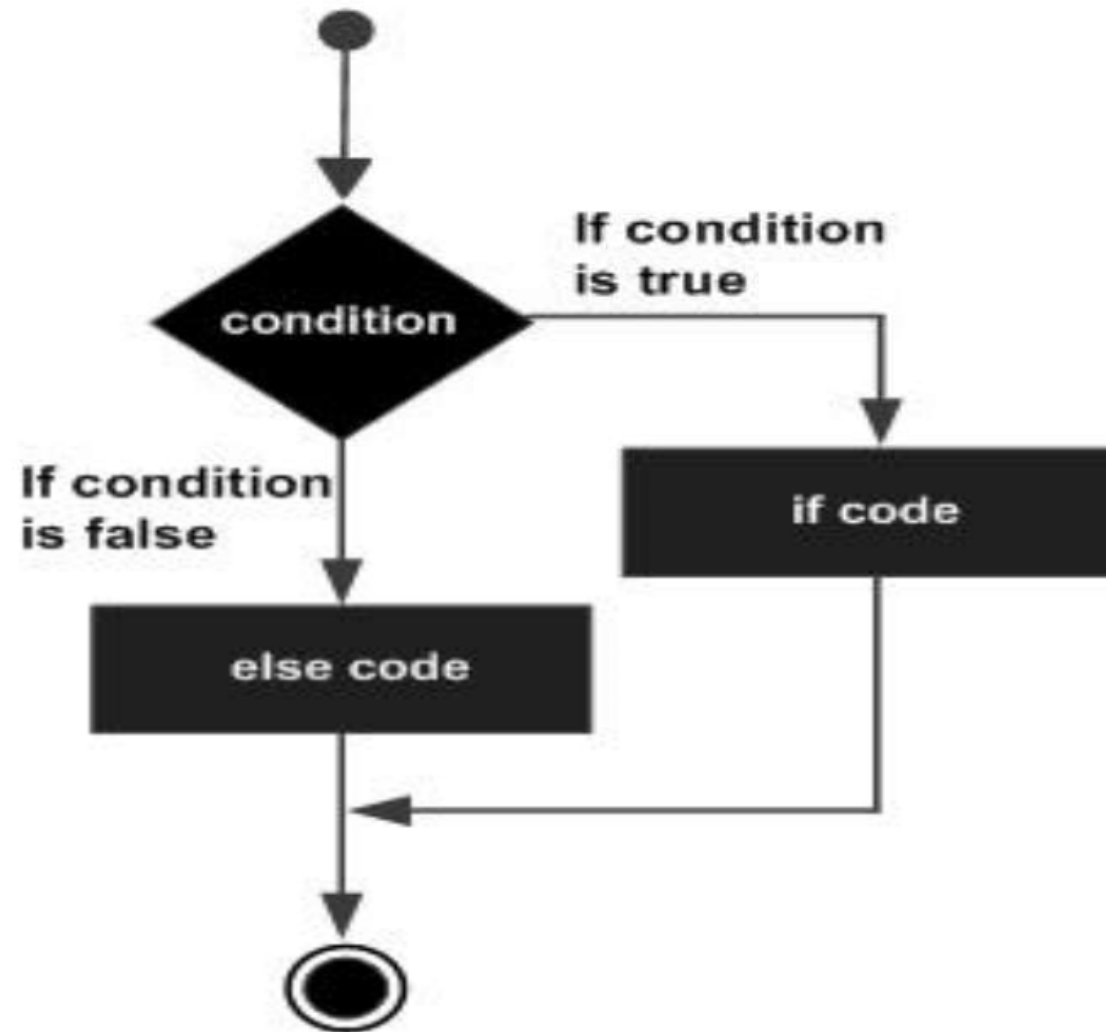
Instructions de contrôle: Condition

- ▶ La déclaration **if ... else** est utilisé si le programmeur veut exécuter certaines déclarations lorsque l'expression de **test est vrai** et exécuter un autre état si l'expression de **test est faux**.

- ▶ **Syntaxe:**

```
if (test expression) {  
    instruction 1;  
    .....;  
    instruction n;  
}  
  
else {  
    instructionelse 1;  
    .....;  
    instructionelse n;  
}
```


Instructions de contrôle: Condition



Instructions de contrôle: Condition

► Exemple :

```
1  *****
2  Prorgamme qui test si la valeur saisie par utilisateur
3  est un nombre pair ou un nombre impair
4  *****
5
6  #include <stdio.h>
7
8  void main()
9  {
10     int a;                //Declration de Variables
11     printf("Entrer la valeur de a :"); // Affichge de message sur ecran pour inviter utilisateur pour saisir La valeur de a
12     scanf("%d",&a);        // Lecture aprtir de Clavier
13
14     if(a%2==0)             // La condition avec if entre ()
15     | printf(" La valeur %d est pair  \n",a);    // La condition avec if entre (Le reste de La division sur 2=0) est vrai
16     else                  // La condition avec if entre () est fause  reste de La division sur2#0
17     | printf(" La valeur %d est impair \n ",a);  // \n est utilise pour revenir a La ligne
18 }
19
```

► Exécution :

```
Entrer la valeur de a : 12
La valeur 12 est pair
```

```
Entrer la valeur de a : 3
La valeur 3 est impair
```

Instructions de contrôle: Condition

► Exemple :

```
1  /******  
2      Prorgamme qui test si La valeur saisie par utilisateur  
3      est un positif , negatif ou nul  
4  *****/  
5  
6  #include <stdio.h>  
7  
8  void main()  
9  {  
10     int a;                //Declration de Variables  
11     printf("Entrer la valeur de a :"); // Affichge de message sur ecran pour inviter utilisateur pour saisir la valeur de a  
12     scanf("%d",&a);        // Lecture aprtir de Clavier  
13  
14     if(a==0)               // La condition avec if entre ()  
15     |   printf(" La valeurd est Nulle \n"); // La condition avec if entre (=0) est vrai  
16     else                   // La condition avec if entre () est fause don il reste deux cas  
17     |   if(a>0)             // La condition avec if entre (>0) est vrai  
18     |   |   printf(" La valeur %d est Positive \n ",a); // \n est utilise pour revenir a la ligne  
19     |   |   else           // Les deux conditions (>0) (=0) sont fausses  
20     |   |   |   printf(" La valeur %d est negative \n ",a);  
21 }
```

► Exécution :

```
Entrer la valeur de a : -3  
La valeur -3 est negative
```

```
Entrer la valeur de a : 0  
La valeurd est Nulle
```

```
Entrer la valeur de a : 4  
La valeur 4 est Positive
```

Instructions de contrôle: Condition

► if(variable)

► Exemple : if(a) a!=0

```
#include <stdio.h>

int main()
{
    int a=3;

    if(a) { printf(" a n'est pas nul");
    }
    return 0;
}
```

► Exécution :

a n'est pas nul

► Exemple : if(!a) a==0

```
#include <stdio.h>

int main()
{
    int a=0;

    if(!a) { printf(" a est nul");
    }
    return 0;
}
```

a est nul

Instructions de contrôle: Branchement

- ▶ Pour éviter les imbrications d'instructions if, le C possède une instruction qui crée une table de branchement : c'est l'instruction **switch**.
- ▶ La syntaxe du **switch** est résumée dans l'encart suivant :

```
switch (expression) {  
    case value1 :  
        inst 10; inst 11; inst 12;  
    case value2 :  
        inst 20; inst 21; inst 22;  
    .....  
    case valueN :  
        inst N0; inst N1; inst N2;  
    default :  
        inst 30 inst 31 inst 32  
}
```

Instructions de contrôle: Condition

► Exemple :

```
1- /******  
2-      Prorgamme LES SAISONS  
3-      *****/  
4-  
5- #include <stdio.h>  
6-  
7- void main()  
8- {  
9-     int a;                //Declration de Variables  
10-    printf("Entrer la valeur de saison (entre 1 et 12) :"); // inviter utilisateur pour saisir la valeur de a entre 1 et 12  
11-    scanf("%d",&a);        // Lecture aprtir de Clavier  
12-  
13-    switch (a)             // Test sur La valeur de a saisie  
14-    {  
15-        case 1 ... 3 : printf("Hiver");  
16-                       break;          // break pour sortir du bloc Switch  
17-        case 4 ... 6 : printf("Printemps");  
18-                       break;  
19-        case 7 ... 9 : printf("Ete");  
20-                       break;  
21-        case 10 ... 12 : printf("Automne");  
22-                        break;  
23-        default:      printf("Erreur dans la saisie");    // Le cas de saisie d'une valeur hors [1..12]  
24-                       break;  
25-    }  
26- }
```

► Exécution :

```
Entrer la valeur de saison (entre 1 et 12) : 4  
Printemps
```

```
Entrer la valeur de saison (entre 1 et 12) : 4  
Printemps
```

Contenu :

- ▶ **Séance 2 Operations :**
 - ▶ **Opérateurs** : binaire et unaire
 - ▶ **Conditions**
 - ▶ **Boucles itératives**
 - ▶ **Ruptures de séquence**

Instructions itératives : boucles

- ▶ Le **processus itératif** est une séquence d'instructions destinée à être exécutée **plusieurs fois** et autant de fois qu'on peut en avoir besoin. C'est aussi une exécution de la séquence.
- ▶ Les instructions itératives sont commandées par trois types de boucles :

while
for
do while

Instructions itératives : boucles

► Instructions itératives: **While**

- Syntaxe de la boucle **While** en algorithme :

Tantque **<condition logique est vraie>** faire
 <bloc d'instructions>
FinTantque

- **Syntaxe** de la boucle **While** en C:

```
exp1;  
while(exp2) {  
    inst 1;  
    inst n;  
    exp3;  
}
```

Instructions itératives : boucles

► Instructions itératives: **While**

► Exemple :

```
1  /*****
2  Programme qui permet d'afficher les nombres de 0 à N
3  *****/
4  #include <stdio.h>
5  int main()
6  {
7      int N,i ;           // i sera utilise comme compteur
8
9      printf("Entrez la valeur N que vous voulez afficher de 0 ... N : ");
10     scanf("%d",&N);
11
12     i=0;                 // initialisation de i avec 0
13     while(i<=N)          // condition a respecter est que i ne depasse pas N
14     {
15         printf(" %d \t",i);
16         i++;             // Incrementation de i =i+1
17     }
18
19
20
21
22     return 0;
23 }
24
```

► Exécution :

```
Entrez la valeur N que vous voulez afficher de 0 ... N : 9
0      1      2      3      4      5      6      7      8      9
```

Instructions itératives : boucles

► Instructions itératives: **do While**

- A l'inverse du **while**, le **do while** place son test en fin d'exécution, d'où au moins une exécution.
- Il ressemble aux **REPEAT UNTIL** d'ALGOL.
- L'organigramme correspond à celui du **while** mais avec le **test en fin** plutôt **qu'en début**, ce qui assure que l'instruction est réalisée **au moins une fois**.
- **Syntaxe** de la boucle **While** en C:

```
exp1;  
do {  
    inst 1;  
    inst n;  
    exp3;  
} while(exp2)
```

Instructions itératives : boucles

► Instructions itératives: **do While**

► Exemple :

```
1  /*****
2  Programme qui permet d'afficher les nombres de 0 à N
3  *****/
4  #include <stdio.h>
5  int main()
6  {
7      int N,i ;           // i sera utilisé comme compteur
8
9      printf("Entrez la valeur N que vous voulez afficher de 0 ... N : ");
10     scanf("%d",&N);
11
12     i=0;                 // initialisation de i avec 0
13     // condition à respecter est que i ne dépasse pas N
14     do {
15         printf(" %d \t",i);
16         i++;             // Incrementation de i =i+1
17     }while(i<=N);
18     return 0;
19 }
20
21
```

► Exécution :

```
Entrez la valeur N que vous voulez afficher de 0 ... N : 9
0      1      2      3      4      5      6      7      8      9
```

Instructions itératives : boucles

► Instructions itératives: **do While**

► Exemple :

```
1  /*****
2  Programme qui permet d'afficher les nombres de 0 à N
3  *****/
4  #include <stdio.h>
5  int main()
6  {
7      int N,i ;           // i sera utilisé comme compteur
8
9      printf("Entrez la valeur N que vous voulez afficher de 0 ... N : ");
10     scanf("%d",&N);
11
12     i=0;                 // initialisation de i avec 0
13     // condition à respecter est que i ne dépasse pas N
14     do {
15         printf(" %d \t",i);
16         i++;             // Incrementation de i =i+1
17     }while(i<=N);
18     return 0;
19 }
20
21
```

► Exécution : Entrez la valeur N que vous voulez afficher de 0 ... N : -2
0

Instructions itératives : boucles

► Instructions itératives : **for**

for(exp1;exp2 ;exp3)

- Le for s'utilise avec trois expressions, séparées par des points virgules, qui peuvent être vides :

- 1) **l'expression expr1** est réalisée une seule fois lors de l'entrée dans la boucle: **expression d'initialisation** ;
- 2) **l'expression expr2** est **la condition d'exécution** de l'instruction . Elle est testée à chaque itération, y compris la première. Si l'expression expr2 prend la valeur vraie l'instruction contrôlée par le for est exécutée, sinon la boucle se termine
- 3) **l'expression expr3** contrôle **l'avancement de la boucle**. Elle permet de manière générale de calculer la prochaine valeur avec laquelle la condition de passage va être re-testée, elle est exécutée après l'instruction à chaque itération avant le nouveau test de passage

Instructions itératives : boucles

► Instructions itératives: **for**

► Exemple :

```
1  /*****
2  Programme qui permet d'afficher Les nombres pairs de 0 a N
3  *****/
4  #include <stdio.h>
5  int main()
6  {
7      int N,i ;          // i sera utilise comme compteur
8
9      printf("Entrez la valeur N que vous voulez afficher de 0 ... N : ");
10     scanf("%d",&N);
11
12     for(i=0;i<=N;i=i+2) //initialisation i=0 ; condition i<=N; Incrementation i =i+2
13     {
14         printf(" %d \t",i);
15     }
16     return 0;
17 }
18 }
```

► Exécution :

```
Entrez la valeur N que vous voulez afficher de 0 ... N : 9
0          2          4          6          8
```

Instructions itératives : boucles

- ▶ Instructions itératives: **for**

- ▶ Exemple :

```
for(i=0; i<5; i++)  
{  
    printf( " %d ", i);  
    i++;  
}
```


Contenu :

- ▶ **Séance 2 Operations :**
 - ▶ **Opérateurs** : binaire et unaire
 - ▶ **Conditions**
 - ▶ **Boucles itératives**
 - ▶ **Ruptures de séquence**

Choix de boucles

- ▶ Utiliser la boucle **for** quand le nombre d'itérations (de répétitions) est connu avant d'entrer dans la boucle :

```
for( initialisations ; Conditions; Incrémentation/ Décrémentations )  
{  
    Instructions;  
}
```

- ▶ Sinon, utiliser une structure boucle conditionnelle

- ▶ Utiliser la boucle **do...while** quand les instructions doivent être effectuées au moins une fois

```
initialisations ;  
do{Incrémentation/ Décrémentations  
    Instructions ;  
}while(Conditions) ;
```

- ▶ Sinon, utiliser la forme **while**

```
initialisations ;  
while(Conditions) {  
    Incrémentation/ Décrémentations;  
    Instructions ;  
};
```

Choix de boucles

- ▶ **Exemple** : Suite arithmétique
 - ▶ *Ecrire un Programme en langage qui donne la valeur de U_n*

$$U_0 = 0, U_{n+1} = U_n + 3.$$

Choix de boucles

► Exemple : Suite arithmétique

```
1  #include <stdio.h>
2  int main()
3  {
4      printf("\n*****La Suite Un+1=Un + 3 *****");
5      int N,i;
6      int S=0; // U0=0
7      printf("\n Entrer un Nombre Positif Non Nul : ");
8      scanf("%d",&N);
9      for(i=0;i<N;++i)
10         S=S+3;
11     printf("  U%d =  %d  ",N,S);
12     return 0;
13 }
14 //  i=0      1  2  3  4  5
15 //  Ui=0     3  6  9  12 15
```

► Exécution:

```
*****La Suite Un+1=Un + 3 *****
Entrer un Nombre Positif Non Nul : 3
U3 =  9
```

Choix de boucles : Exemple

► Boucle do while

► Exemple :

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int N;
6      do{
7          printf("Entrer un Nombre Positif Non Nul : ");
8          scanf("%d",&N);
9      }while(N<=0);
10
11     printf("\n Bienvenu dans la suite du programme");
12     return 0;
13 }
```

► Exécution:

```
Entrer un Nombre Positif Non Nul : -3
Entrer un Nombre Positif Non Nul : -19
Entrer un Nombre Positif Non Nul : -40
Entrer un Nombre Positif Non Nul : 0
Entrer un Nombre Positif Non Nul : 13
```

Choix de boucles : Exemple

► Boucle While

► Exemple :

```
1  #include <stdio.h>
2  int main()
3  {
4      int N=-1;
5      while(N<=0){
6          printf("Entrer un Nombre Positif Non Nul : ");
7          scanf("%d",&N);
8      }
9      printf("\n Bienvenu dans la suite du programme");
10     return 0;
11 }
```

► Exécution:

```
Entrer un Nombre Positif Non Nul : 0
Entrer un Nombre Positif Non Nul : -1
Entrer un Nombre Positif Non Nul : -12
Entrer un Nombre Positif Non Nul : 3

Bienvenu dans la suite du programme
```

Choix de boucles: Exemple

► Boucle While

► Exemple :

```
1  #include <stdio.h>
2  int main()
3  {
4      int N,i;
5      for(i=0;i<1;++i){
6          printf("Entrer un Nombre Positif Non Nul : ");
7          scanf("%d",&N);
8          if(N<=0) i--;
9      }
10     printf("\n Bienvenu dans la suite du programme");
11     return 0;
12 }
```

► Exécution:

```
Entrer un Nombre Positif Non Nul : -12
Entrer un Nombre Positif Non Nul : -3
Entrer un Nombre Positif Non Nul : 0
Entrer un Nombre Positif Non Nul : 4

Bienvenu dans la suite du programme
```

Choix de boucles

► Exercice

```
#include <stdio.h>
int main()
{
    int i,j;
    for( i=0,j=0;i<4,j<10;j++,i++)
        printf("i=%d j=%d \n",i,j);

    return 0;
}
```

```
#include <stdio.h>
int main()
{
    int i,j;
    for( i=0,j=10;i<4,j>0;j--,i++)
        printf("i=%d j=%d \n",i,j);

    return 0;
}
```

```
#include <stdio.h>
int main()
{
    int i,j;
    for( i=0;i<4;i++)
        for( j=0;j<4;j++)
            printf("i=%d j=%d \n",i,j);

    return 0;
}
```


Choix de boucles

► Exercice

```
#include <stdio.h>
int main()
{
    int i,j;
    for( i=0,j=0;i<4,j<10;j++,i++)
        printf("i=%d j=%d \n",i,j);

    return 0;
}
```

```
i=0 j=0
i=1 j=1
i=2 j=2
i=3 j=3
i=4 j=4
i=5 j=5
i=6 j=6
i=7 j=7
i=8 j=8
i=9 j=9
```

```
#include <stdio.h>
int main()
{
    int i,j;
    for( i=0,j=10;i<4,j>0;j--,i++)
        printf("i=%d j=%d \n",i,j);

    return 0;
}
```

```
i=0 j=10
i=1 j=9
i=2 j=8
i=3 j=7
i=4 j=6
i=5 j=5
i=6 j=4
i=7 j=3
i=8 j=2
i=9 j=1
```

```
#include <stdio.h>
int main()
{
    int i,j;
    for( i=0;i<4;i++)
        for( j=0;j<4;j++)
            printf("i=%d j=%d \n",i,j);

    return 0;
}
```

```
i=0 j=0
i=0 j=1
i=0 j=2
i=0 j=3
i=1 j=0
i=1 j=1
i=1 j=2
i=1 j=3
i=2 j=0
i=2 j=1
i=2 j=2
i=2 j=3
i=3 j=0
i=3 j=1
i=3 j=2
i=3 j=3
```

Ruptures de séquence

► Ruptures de séquence

- Dans le cas où une boucle commande l'exécution d'un bloc d'instructions, il peut être intéressant de vouloir sortir de cette boucle alors que la condition de passage est encore valide.
- Ce type d'opération est appelé une **rupture de séquence**.
- Les **ruptures de séquence** sont utilisées lorsque des conditions multiples peuvent conditionner l'exécution d'un ensemble d'instructions.
- Les ruptures de séquence sont réalisées par quatre instructions qui correspondent à leur niveau de travail :

continue

break

goto

return

► Ruptures de Programme : **exit()** / **<stdlib.h>**
